AgentCF: Collaborative Learning with Autonomous Language Agents for Recommender Systems

Junjie Zhang
junjie.zhang@ruc.edu.cn
Gaoling School of Artificial Intelligence, Renmin University of China, Beijing, China

Yupeng Hou
yphou@ucsd.edu
UC San Diego
San Diego, California, USA

Ruobing Xie
xrbsnowing@163.com
Tencent, Beijing, China

Wenqi Sun
swq27@126.com
Gaoling School of Artificial Intelligence, Renmin University of China, Beijing, China

Julian McAuley
jmcauley@ucsd.edu
UC San Diego
San Diego, California, USA

Wayne Xin Zhao✉
batmanfly@gmail.com
Gaoling School of Artificial Intelligence, Renmin University of China, Beijing, China

Leyu Lin
goshawklin@tencent.com
Tencent, Beijing, China

Ji-Rong Wen†
jrwen@ruc.edu.cn
Gaoling School of Artificial Intelligence, Renmin University of China, Beijing, China

ABSTRACT

Recently, there has been an emergence of employing LLM-powered agents as believable human proxies, based on their remarkable decision-making capability. However, existing studies mainly focus on simulating human dialogue. Human non-verbal behaviors, such as item clicking in recommender systems, although implicitly exhibiting user preferences and could enhance the modeling of users, have not been deeply explored. The main reasons lie in the gap between language modeling and behavior modeling, as well as the incomprehension of LLMs about user-item relations.

To address this issue, we propose **AgentCF** for simulating user-item interactions in recommender systems through agent-based collaborative filtering. We creatively consider not only users but also items as agents, and develop a collaborative learning approach that optimizes both kinds of agents together. Specifically, at each time step, we first prompt the user and item agents to interact autonomously. Then, based on the disparities between the agents' decisions and real-world interaction records, user and item agents are prompted to reflect on and adjust the misleading simulations collaboratively, thereby modeling their two-sided relations. The optimized agents can also propagate their preferences to other agents in subsequent interactions, implicitly capturing the collaborative filtering idea. Overall, the optimized agents exhibit diverse interaction behaviors within our framework, including user-item, user-user, item-item, and collective interactions. The results show that these agents can demonstrate personalized behaviors akin to those of real-world individuals, sparking the development of next-generation user behavior simulation.

CCS CONCEPTS

• **Information systems → Recommender systems**.

KEYWORDS

Agents, Large Language Models, Collaborative Learning

ACM Reference Format:
Junjie Zhang, Yupeng Hou, Ruobing Xie, Wenqi Sun, Julian McAuley, Wayne Xin Zhao, Leyu Lin and Ji-Rong Wen. 2024. AgentCF: Collaborative Learning with Autonomous Language Agents for Recommender Systems. In *Proceedings of the ACM Web Conference 2024 (WWW '24), May 13–17, 2024, Singapore, Singapore.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3589334.3645537

1 INTRODUCTION

With the recent advancements in large language models (LLMs) [4, 56], LLM-powered agents demonstrate impressive capabilities in autonomous interaction and decision making [21, 27, 41, 56]. By scaling their numbers, these agents exhibit the emergence of human-like behaviors at both the individual and population levels [11, 34]. This observation highlights the potential of employing LLM-powered agents to simulate human social behaviors, such as daily lives in Smallville [34] and software development [35].

Typically, existing studies primarily focus on simulating human dialogue using semantic knowledge of LLMs. However, in addition to dialogue, real-world human behaviors also involve non-verbal

✉ Corresponding author: Xin Zhao.
† Ji-Rong Wen is also with School of Information, Renmin University of China.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
WWW '24, May 13–17, 2024, Singapore, Singapore.
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0171-9/24/05
https://doi.org/10.1145/3589334.3645537

aspects like user-item interactions in recommender systems, which implicitly reflect user preferences and have the potential to facilitate personalized user modeling. To simulate such behaviors, some work verbalizes these interaction records into natural language text to prompt LLMs [41, 44]. Nevertheless, we argue that this method struggles to capture the underlying behavioral patterns of these interactions, due to the gap between universal language modeling and personalized behavior modeling. For example, shoppers who buy diapers on Friday also tend to buy beer [32]. This behavioral pattern can be effectively captured by collaborative filtering models [14, 37], but confuses LLMs as the two items are semantically unrelated. Therefore, it is crucial to develop methods to better characterize human behaviors in LLM-powered simulations.

Focused on this research topic, in this work, we take recommender systems, a common web application, as the testbed to study the simulation of user-item interactions with LLM-power agents. Typically, in addition to directly prompting LLMs based on user historical interactions [8, 42], there are also some studies that employ agents to perform recommendations, by introducing specialized tools and planning strategies [18, 46]. Especially, RecAgent [44] develops an LLM-based simulator, regarding users as agents. However, these studies mainly focus on characterizing user-side behaviors using universal LLMs, neglecting the effect of item-side modeling in this two-sided interaction process. In recommender systems, users possess personalized preferences for items, while items have potential adopters. Modeling the two-sided relations between users and items is crucial for enhancing personalized recommendations [14].

Considering these issues, our solution is inspired by the *collaborative learning* idea from previous recommendation models (*e.g.,* BPR [37] and NCF [14]). In these methods, users and items are modeled as equally important parts and the recommender is essentially a preference function that measures user-item affinity based on the two sides. During the optimization process, the parameters related to users and items are collaboratively optimized, making the recommender better fit interaction records. Therefore, to facilitate the comprehension of user-item relations by LLM-powered agents, our idea is to simulate both *user agents* and *item agents*, and mimic the optimization process of traditional recommenders. The merits of such a method are twofold. Firstly, it can model the user-item interaction by autonomous interaction between user and item agents, instead of verbalizing the interaction as plain text [44]. Secondly, it enables mutual optimization of user and item agents to capture their two-sided relations, rather than simulating user behaviors individually. We are also aware of several studies, such as Reflexion [39] and Self-Refine [29], optimize general-purpose agents for better decision-making in downstream tasks. However, these methods are task-focused or user-focused, optimizing agents through their own exploration, which is essentially *self-learning* rather than *collaborative learning* emphasized in our approach.

To this end, we propose the agent-based collaborative filtering approach, named **AgentCF**. Unlike previous studies that mainly focus on user simulation, our approach considers not only users but also items as agents. Both kinds of agents are equipped with memory modules, maintaining the simulated preferences and tastes of potential adopters. This involves their intrinsic features as well as acquired behavioral information. As the key of our approach, we collaboratively optimize the user agents and item agents, leveraging the remarkable decision-making and reflection abilities of LLMs. At each step, we prompt user and item agents to autonomously interact, thereby exploring whether these simulated agents can make consistent decisions with real-world interaction records. Then, based on the feedback obtained from these interactions, we design a collaborative reflection mechanism that enables user agents and item agents to reflect on and adjust their memory in a mutual manner. In this way, the simulated preferences of user agents and item agents mutually aggregate, and can be propagated to other agents in subsequent interactions. Thus, it implicitly models the collaborative filtering idea through interactions.

We evaluate our proposed approach through extensive experiments on real-world datasets. Results show that our approach effectively simulates user-item interactions, achieving promising performance on recommendation tasks compared to several classical recommendation models and LLM-based recommenders. Moreover, within our framework, user and item agents engage in free communication, creating various types of interactions (user-user, item-item, and collective), wherein they exhibit personalized behaviors similar to those of real-world individuals. The results also spark the development of next-generation user behavior simulation. The main contributions of this work are as follows:

- We consider both users and items as agents, and develop a user-item interaction simulation approach for recommender systems, emphasizing the modeling of two-sided interaction relations.
- We collaboratively optimize the user and item agents, design a collaborative reflection mechanism, and employ it to achieve mutual update of user and item memory.
- Extensive experiments demonstrate the effectiveness of our approach in simulating personalized interactions.

2 METHODOLOGY

In this section, we present the proposed agent-based collaborative filtering approach, named **AgentCF**. Our approach enables user agents and item agents to collaboratively learn from user-item interactions in recommender systems. The overall framework of our proposed AgentCF is depicted in Figure 1.

2.1 Preliminaries

To ease the understanding of our approach, we first describe the traditional recommendation setting, and then introduce our task setting and the overview of our approach.

Traditional Recommendation Setting. In a recommender system, there exists a set of users $\mathcal{U} = \{u\}$, a set of items $\mathcal{I} = \{i\}$, and a set of their interaction records (often grouped by users in chronological order), denoted by $\mathcal{D} = \{\langle u, i\rangle\}$. To conduct recommendation, a preference function $f(u, i)$ is built to capture the preference degree of user u over item i, where an interacted item (*i.e.,* positive item, denoted by i^+) would receive a higher score than an item that the user does not interact with (*i.e.,* negative item, denoted by i^-). For example, BPR [37] builds a personalized pairwise ranking model that compares the preference over two candidate items, and NCF [14] employs neural networks to fit user-item interaction relationship. Subsequently, the learning of preference function $f(u, i)$ can be converted into a standard gradient-based function fitting problem based on training data $\mathcal{D} = \{\langle u, i\rangle\}$ in machine learning [2].

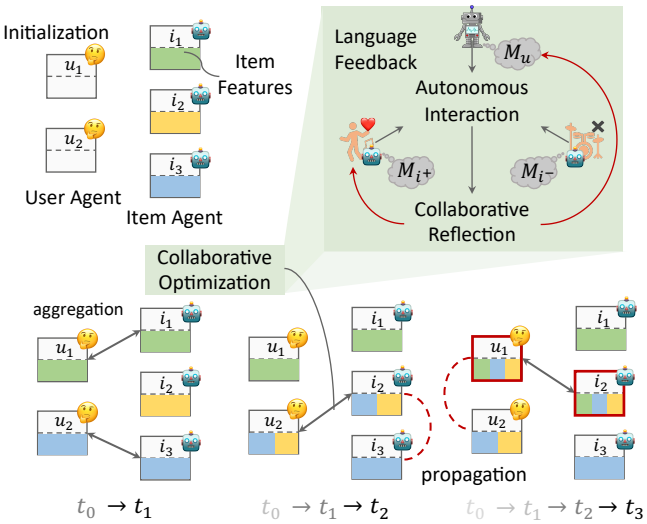

Figure 1: The overall framework of AgentCF: (1) The user and item agents are first prompted to autonomously interact. (2) These agents adjust the misconceptions in their memory, by reflecting on the disparities between their decisions and real-world interactions. In this process, the simulated preferences of user and item agents aggregate and can propagate to other agents in subsequent interactions.

Our Task Setting. In this work, we follow the traditional recommendation setting above and incorporate LLM-powered agents into recommender systems. Different from prior studies [34, 44] only considering task or user agents, *item agents* are also included in our work and will be a crucial part of developing our approach. Notably, we slightly reuse the notations, and use u and i to denote a user agent and an item agent, respectively. Specifically, a user agent u is expected to capture the preference of the corresponding real user, and an item agent i is expected to reflect the corresponding item characteristics and potential adopters' preferences. Following such a setting, the original user-item interaction will be simulated by autonomous interaction between the user and item agents. In our work, we consider a *ranking* task that ranks a candidate list $\{c_1, \cdots, c_n\}$ for a user agent u based on LLMs: $f_{LLM}(u, \{c_1, \cdots, c_n\})$. Different from traditional recommendation models, the LLM that implements $f_{LLM}(\cdot)$ will be fixed during the optimization process.

Overview of AgentCF. To implement the agents, the memory mechanism (*i.e.,* storing the past states, actions, and contexts [43]) and the reflection mechanism (*i.e.,* revising the agents' states or recognitions [33, 39]) have been widely explored in various task scenarios. However, existing work often employs a task-focused (*e.g.,* ReAct [52]) or user-oriented simulation approach (*e.g.,* RecAgent [44]), and the objects (*e.g.,* the items in recommender systems) involved in this process are not explicitly considered. Since the recommendation task essentially needs to model the two-sided interaction relation between users and items, we argue that item agents are important to simulate the real interaction scenario in recommender systems. Without incorporating item agents, it is difficult to explicitly model the essential idea "*like alike*" (*i.e.,* like the items that are adopted by similar users) in collaborative filtering [3]. Notably, our approach has two technical contributions in adapting autonomous language agents for recommender systems:

- *Collaborative memory-based optimization.* Instead of simply prompting LLMs with user's historical interactions [8, 28] , we aim to refine (without gradient update) both the simulated user and item agents through their mutual interactions. At each time step, these agents are prompted to conduct autonomous interactions, and then collaboratively reflect on the disparities between their decisions and real-world interaction records. In this way, the simulated user and item agents can accordingly adjust or update their memories, enabling them to better fit the real-world interaction behaviors.
- *Implicit preference propogation.* Different from prior studies [35, 44], a notable feature of our approach is that we update the memory of both users and item agents for each interaction record. This enables the item memory to be injected into the preferences of users who interact with it. When new interactions occur, subsequent users will be also informed about the preference of previous adopters of this item. The same process can also be applied to the user side. In this case, our approach essentially models the collaborative filtering idea by propagating preferences from user-item interactions.

2.2 Collaborative Agent Optimization

In this part, we present the collaborative optimization process that is developed based on both user and item agents, and discuss how it relates to the classic collaborative filtering approach.

2.2.1 Memory Design. To specialize the LLM-powered agents for recommender systems, different from prior studies [34, 44], we equip both user and item agents with memory modules, maintaining their intrinsic features and collaborative information.

User Memory. For user agents, the memory module aims to store various kinds of useful information that reflects the user preferences. Since real-world user preferences often change dynamically, we equip each user agent u with short-term memory M_u^s and long-term memory M_u^l. Especially, short-term memory is a natural language text that describes the recently updated preference of the user agent, which can be initialized with their general preferences like "*I enjoy listening to CDs*". Furthermore, long-term memory is a pool of historical preference texts that can store the evolving process of user preference. When engaged in a new interaction, user agents can directly access their short-term memory, while retrieving relevant information from their long-term memory.

Item Memory. For item agents, we equip them with adjustable memory modules to record information about their own characteristics as well as the preferences of their adopters, *e.g.,* a rock CD may be ideal for energetic guitar-driven enthusiasts. However, unlike user memory, we only equip each item agent i with a unified memory module M_i, since item information is relatively stable through time. Item memory can be initialized by their identity information, such as titles and categories, and will be continuously updated with user preferences when making new interactions. This process can largely complete the global characteristics of an item in real-world systems and enable the propagation of preference information, which is the key to collaborative learning in our approach.

2.2.2 Autonomous Interactions for Contrastive Item Selection. Given the initialized agents, our task is to optimize these agents and enable them to simulate real-world user-item interactions. To achieve this, we first explore the behavior alignment between these simulated agents and real-world individuals, so as to provide feedback for updating these agents. This involves testing whether the agents can make consistent decisions with real-world interaction records when selecting candidates. We can also consider this process as the "*forward computation*" of the recommender models (*e.g.,* BPR [37]) when comparing the preference over two items by a user.

Specifically, we employ the real user behavioral sequences (arranged in chronological order) as "training data". At each step of these interactions, we present a contrastive pair of both a positive item i^+ and a negative item i^- as candidates for user agents to select from. Intuitively, agents lacking personalization mainly rely on commonsense knowledge to select popular candidates and those positioned higher in the display list for interaction. To increase the discrimination difficulty, we deliberately introduce popular bias and position bias in candidate selection: we sample a negative item with high popularity in the dataset and then place it before the positive item in the selection list. Then, the user agent u is tasked with selecting an item i^o from the candidates and providing explanations y_{exp} for this choice, by collaboratively considering the preference from its memory and the features from the memories of candidates:

$$i^o = f_{LLM}(M_u;\ M_{i^-};\ M_{i^+}) \tag{1}$$

$$y_{exp} = \text{Prompt}_{LLM}(i^o;\ M_u;\ M_{i^-};\ M_{i^+}) \tag{2}$$

2.2.3 Collaborative Reflection and Memory Update. The above process has enabled agents to simulate the interaction behaviors. Next, we derive the feedback signal by comparing the agents' decisions with real-world interaction data, and employ it to collaboratively optimize the involved user agent and item agents, serving as the "backward update" part of traditional recommenders. Since LLMs are fixed in this work, there is no explicit gradient-based learning process as in traditional recommender models [37]. Thus, we mainly update the associated memories of user and item agents.

Based on the agent's decision i^o and explanation y_{exp} generated above, we prompt both user and item agents to reflect on and adjust the misconceptions in their simulated preferences and features. We refer to this process as *collaborative reflection*, as it emphasizes the collaborative learning of the user-item relations, by performing reflection based on both user and item memories. It is different from task-specific reflection (*e.g.,* ReAct [52]) and user-oriented reflection (*e.g.,* RecAgent [44]), which construct the *self-reflection* only from the perspective of the task solver or user. Specifically, if a user agent makes the right choice that aligns with the real behavior, we inform it about the correctness and store the related interaction information in its memory. For an incorrect choice, we introduce the following collaborative reflection mechanism to revise agents' memories and behaviors accordingly, enabling user agents to simulate real interaction behaviors and the involved item agents to align with the adopter's preferences:

$$M_u^{s\prime} \leftarrow \text{Reflection}^u(i^o;\ y_{exp};\ M_u;\ M_{i^-};\ M_{i^+}), \tag{3}$$

$$M_i' \leftarrow \text{Reflection}^i(i^o;\ y_{exp};\ M_u;\ M_{i^-};\ M_{i^+}) \tag{4}$$

$$M_u^{l\prime} \leftarrow \text{Append}(M_u^l;\ M_u^s), \tag{5}$$

where $M_u^{s\prime}$ denote the reflected short-term memory of user u and M_i' denote the reflected memory of item i. $M_u^{l\prime}$ is the updated long-term memory of user u, which appends the user's previous short-term memory and can be retrieved when making a new interaction. Note that we do not modify the memory of the negative item agent, because we find that LLMs tend to over-complain the negative item agent's drawbacks, disregarding the fact that it may also be attractive for other users. From Equation (3) and (4) (a similar prompting method in Equation (2)), we can see that the reflection is actually derived based on the memories of user agent u, positive item agent i^+, and negative item agent i^-. Therefore, such a reflection mechanism enables user and item agents to better understand their two-sided interaction relations by effectively discriminating between positive instances and negative instances.

At each step of interaction (*i.e.,* optimization), we iterate the selection (Section 2.2.2) and the collaborative reflection process (Section 2.2.3) until user agents make choices consistent with those of real users or reach the maximum round of iterations. In essence, throughout the collaborative optimization process, the simulated user and item agents follow the decision-making process of real-world individuals, progressing step by step. This makes agents become more personalized by learning from interactions, improving the simulation of user-item interactions. The implementation details and prompts are presented in Appendix A and B.

2.2.4 Connection with Classical Recommendation Models. In the field of recommender systems, various models such as BPR [37] and NCF [14] are developed. In general, these models set parameters (typically hidden embeddings) to represent user preferences and item characteristics, and optimize them to fit user-item interaction records through two stages, namely *forward preference evaluation* (estimating preference score) and *backward parameter update* (performing gradient update). In this way, users or items that display similar interaction behaviors can have similar representations in parameter space, thus capturing the collaborative filtering idea.

Our approach mimics the optimization process of traditional recommenders. The user and item memories can be considered as their language-based parameters (*i.e.,* embeddings). The item selection process (Section 2.2.2) and collaborative reflection process (Section 2.2.3) correspond to the forward and backward stages in recommendation models. In our approach, explicit gradient optimization is not employed. Instead, the collaborative reflection plays a similar role of "*semantic gradient*", steering how user and item agents adjust themselves. Moreover, based on the collaborative reflection, the user and item agents mutually interact and aggregate each other's preference information, which is then propagated to new agents in subsequent interactions. Therefore, by establishing **preference propagation** from interactions of user and item agents, our approach incorporates the idea of collaborative filtering.

2.3 Agent Interaction Inference

After the above memory-based optimization, our approach can simulate highly personalized user agents and preference-aware item agents. In this part, we further study how to employ these simulated agents to infer the potential user-item interactions, focusing on the ranking task for a list of candidates $\{c_1, \cdots, c_n\}$.

Basic Prompting Strategy. Since both user and item agents are collaboratively optimized to model their two-sided relations, as the basic prompting form, we directly prompt the LLM with the agents' simulated user preferences and the candidate item features. This enables the LLM to serve as a collaborative recommender:

$$\mathcal{R}_B = f_{LLM}(M_u^s;\ \{M_{c_1}, \cdots,\ M_{c_n}\}) \tag{6}$$

where M_u^s is the short-term memory of user agent u, $\{M_{c_1}, \cdots, M_{c_n}\}$ are the corresponding memories of the candidate item agents, n is the number of candidates, and $\mathcal{R}$ is the ranking result.

Advanced Prompting Strategies. Although short-term memory describes the current preference of a user agent, retrieving their specialized preferences from long-term memory toward candidates can allow them to make more personalized inferences. In addition, when interaction records are sparse and preference propagation is limited, we can further incorporate user historical interactions into prompts, enabling LLMs to serve as sequential recommenders. These two improvement strategies can be denoted as follows:

$$\mathcal{R}_{B+R} = f_{LLM}(M_u^r;\ M_u^s;\ \{M_{c_1}, \ldots,\ M_{c_n}\}) \tag{7}$$

$$\mathcal{R}_{B+H} = f_{LLM}(M_u^s;\ \{M_{i_1}, \ldots,\ M_{i_m}\};\ \{M_{c_1}, \ldots,\ M_{c_n}\}) \tag{8}$$

where M_u^r is the retrieved specialized preference from the long-term memory of user agent u by taking the memories of candidate items as queries, and $\{M_{i_1}, \cdots, M_{i_m}\}$ are the corresponding memories of the m historical interactions with items $\{i_1, \ldots, i_m\}$.

Based on the optimized user agents and item agents, our method can better simulate diverse interaction behaviors observed in the real world, such as user-item interactions, users' social behaviors, and even collective behaviors within recommender systems. Moreover, since we activate items as agents, it enables us to explore the potential feasibility of "inventing" more novel and fascinating interactions among inanimate objects. For example, item-to-item agent interaction can be useful in item cold-start scenarios by spontaneously propagating collected user preference to new items (See Section 3.4.2 for in-depth analysis experiments).

3 EXPERIMENTS

3.1 Experimental Setup

3.1.1 Datasets. Following previous work [15, 16], we conduct experiments on two text-intensive subsets of Amazon review dataset [31]: "CDs and Vinyl" and "Office Products". Due to expensive API calls, we have to further sample subsets from these two datasets. Specifically, considering the effect of data sparsity on collaborative filtering recommenders, we randomly sample two subsets (one dense and one sparse) from each dataset, with each subset containing 100 users, exploring more diverse interaction scenarios. The statistics of the datasets are summarized in Table 1. Note that we use the dense datasets for further analysis experiments, as they better demonstrate the interactions among agents.

3.1.2 Evaluation Metrics. To evaluate the performance, we take NDCG@K as a metric, where K is set to 1, 5 and 10. Following existing studies [16, 57], we employ the *leave-one-out* strategy for evaluation. Specifically, we consider the last item in each historical interaction sequence as the ground-truth item. By adopting the model as a ranker, we rank the target item alongside nine randomly sampled items. To further reduce randomness, we conduct three repetitions of all test instances and report the average results.

Table 1: Statistics of the preprocessed datasets. "Avg.c" is the average number of words in initialized item description text.

Datasets	#Users	#Items	#Inters.	Sparsity	Avg.c
CDs (full)	93,653	64,032	1,178,439	99.98%	8.04
–Sparse	100	704	800	98.86%	7.76
–Dense	100	269	800	97.03%	8.47
Office (full)	86,530	25,842	675,683	99.97%	25.14
–Sparse	100	561	600	98.93%	25.06
–Dense	100	188	600	96.81%	25.48

3.1.3 Baseline Models. We compare the proposed method with the following baseline methods:

• **BPR** [37] leverages matrix factorization to learn the representations of users and items by optimizing the BPR loss.

• **SASRec** [20] captures the sequential patterns of user historical interactions by utilizing a transformer-encoder.

• **Pop** ranks candidates based on their popularity, which is measured by the number of interactions.

• **BM25** [38] ranks candidates according to their textual similarity with user historical interactions.

• **LLMRank** [17] uses the ChatGPT as a zero-shot ranker, considering user sequential interaction histories as conditions.

For our approach [1], we consider three major variants: AgentCF$_B$, AgentCF$_{B+R}$, and AgentCF$_{B+H}$ (corresponding to Equation (6), (7), and (8), respectively). As our simulated user and item agents are optimized on sampled datasets, we compare our method with BPR and SASRec by training them on the sampled datasets, referred to as BPR$_{\text{sample}}$ and SASRec$_{\text{sample}}$. We also report the performance of these two models when trained on the complete dataset for reference, denoted as BPR$_{\text{full}}$ and SASRec$_{\text{full}}$. The Pop model, which relies on statistics to make recommendations, is trained on the complete datasets. Since LLMRank and BM25 are zero-shot models, training them on the complete dataset would be meaningless. Therefore, we directly evaluate their performance on the sampled datasets. Scaling agents for larger datasets is left as future work.

3.2 Overall Performance

We compare our approach with the baseline methods on four datasets and present the results in Table 2. We can find:

(1) Our approach outperforms other baselines in most scenarios, highlighting the effectiveness of collaborative learning for simulating personalized agents. Specifically, the performance achieved by adopting our model as a collaborative filtering recommender (AgentCF$_B$) demonstrates the efficacy of preference propagation in modeling the collaborative filtering idea. In addition, retrieving user agents' specialized preferences toward candidates (AgentCF$_{B+R}$) enables more personalized inferences. In sparse interaction cases (Office dataset), incorporating user historical interactions to enhance the prompt and enabling LLMs to serve as a sequential recommender (AgentCF$_{B+H}$) can make better performance.

[1]The code is available at https://github.com/RUCAIBox/AgentCF.

Table 2: Performance comparison of different models. We highlight the best and the second-best among traditional recommenders trained on sampled datasets, tuning-free models, and our approach, using bold and underlined fonts, respectively.

Method	CDs_{sparse}			CDs_{dense}			$Office_{sparse}$			$Office_{dense}$		
	N@1	N@5	N@10	N@1	N@5	N@10	N@1	N@5	N@10	N@1	N@5	N@10
BPR_{full}	0.1900	0.4902	0.5619	0.3900	0.6784	0.7089	0.1600	0.3548	0.4983	0.5600	0.7218	0.7625
$SASRec_{full}$	0.3300	0.5680	0.6381	0.5800	0.7618	0.7925	0.2500	0.4106	0.5467	0.4700	0.6226	0.6959
BPR_{sample}	0.1300	0.3597	0.4907	0.1300	0.3485	0.4812	0.0100	0.2709	0.4118	0.1200	0.2705	0.4576
$SASRec_{sample}$	0.1900	0.3948	0.5308	0.1300	0.3151	0.4676	0.0700	0.2775	0.4437	**0.3600**	**0.5027**	**0.6137**
Pop	0.1100	0.2802	0.4562	0.0400	0.1504	0.3743	0.1100	0.2553	0.4413	0.0700	0.2273	0.4137
BM25	0.0800	0.3066	0.4584	0.0600	0.2624	0.4325	0.1200	0.2915	0.4693	0.0600	0.3357	0.4540
LLMRank	0.1367	0.3109	0.4715	0.1333	0.3689	0.4946	0.1750	0.3340	0.4728	0.2067	0.3881	0.4928
$AgentCF_B$	0.1900	0.3466	0.5019	0.2067	0.4078	0.5328	0.1650	0.3359	0.4781	0.2067	0.4217	0.5335
$AgentCF_{B+R}$	**0.2300**	**0.4373**	**0.5403**	**0.2333**	0.4142	**0.5405**	0.1900	0.3589	0.5062	0.1933	0.3916	0.5247
$AgentCF_{B+H}$	0.1500	0.4004	0.5115	0.2100	**0.4164**	0.5198	**0.2133**	**0.4379**	**0.5076**	0.1600	0.3986	0.5147

Table 3: Ablation study on two sampled datasets.

Variants	CDs_{dense}		$Office_{dense}$	
	N@1	N@10	N@1	N@10
$AgentCF_B$	**0.2067**	**0.5328**	0.2067	**0.5335**
w/o Auto. Interaction	0.1200	0.4964	0.1733	0.5031
w/o User Agent	0.1100	0.4693	**0.2200**	0.5145
w/o Item Agent	0.1767	0.5128	0.1800	0.5169

(2) Our model exhibits superior or comparable performance to traditional recommendation models when trained on datasets of the same scale (sampled datasets). Furthermore, our model even achieves comparable performance to traditional recommenders trained on full datasets, when dealing with sparse scenarios. Although our model still has room for improvement in other scenarios, it is worth emphasizing that our model is trained using only approximately 0.07% of the complete dataset. These results further demonstrate the generalization capability of our approach.

(3) Existing tuning-free methods, such as Pop, BM25, and LLMRank, exhibit unsatisfactory performance. Specifically, although LLMRank employs ChatGPT to model user preferences by introducing user historical interactions as prompts, similar to conclusions in [17, 28], its performance falls significantly behind that of traditional models. This highlights the gap between user behavior patterns and universal knowledge of LLMs, which hinders the effectiveness of LLMs in capturing user preferences from their behaviors.

3.3 Further Model Analyses

3.3.1 Ablation Study. Our proposed agent-based collaborative filtering approach contains the agents' autonomous interaction part, the optimization of user agents, and the optimization of item agents. To verify the effectiveness of each component, we conduct the ablation study on two dense datasets and record the results in Table 3.

(1) w/o Autonomous Interaction: In this scenario, we directly provide real user interaction records to agents, and prompt them to explore the reasons behind real user's decisions. The performance gap reveals that comparing agents' autonomous interactions with real-world interaction records can generate comprehensive feedback about the misalignment between simulated agents and real-world individuals, which enhances the efficacy of reflection.

(2) w/o User Agent: Here we directly represent each user with their historical interactions, without any memory update. The performance decline indicates that the simple verbalized text of user interactions can not reflect their preferences, emphasizing the efficacy of adjusting agents' memories by enabling them to autonomously simulate real user behaviors. Although prompting LLMs with user historical interactions can yield better results in the Office dataset, we argue that this is due to the dataset's longer item description text, allowing LLMs to effectively serve as sequential recommenders. However, even in this case, stimulating LLM-powered item agents can lead to improved performance, further demonstrating the significance of facilitating items to capture preferences of adopters.

(3) w/o Item Agent: Not optimizing item agents (*i.e.,* representing items with corresponding identity information) also leads to worse performance, highlighting the effectiveness of behavior-involved item-side modeling in capturing two-sided interaction relations between users and items. Notably, item agents play a crucial role in simulating collaborative filtering recommender systems, by updating their memory with user preference information and propagating this information to new agents through interactions.

3.3.2 Performance Copmarison w.r.t. Position and Popularity Bias. In this part, we evaluate our approach's ability to simulate personalized agents, by obliquely examining whether they are susceptible to position and popularity bias in ranking candidates, as general LLMs heavily rely on such common-sense knowledge to make judgments. The results in Figure 2 show that an LLM prompted by user historical interactions (LLMRank) is influenced by these biases. It tends to select popular items and those positioned higher. While our approach, although inevitably affected by these biases, performs enhanced stability. This confirms that our approach simulates personalized user agents, enabling them to rank candidates based on their preferences rather than relying solely on general knowledge.

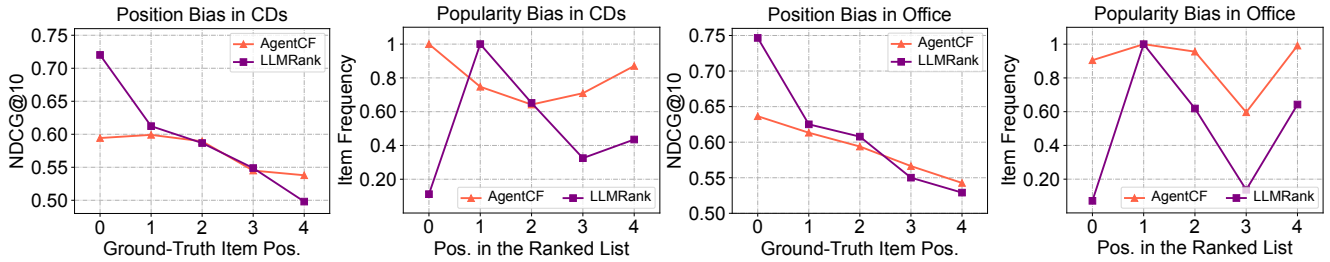

Figure 2: Analysis of whether our approach can simulate personalized agents to mitigate position bias and popularity bias.

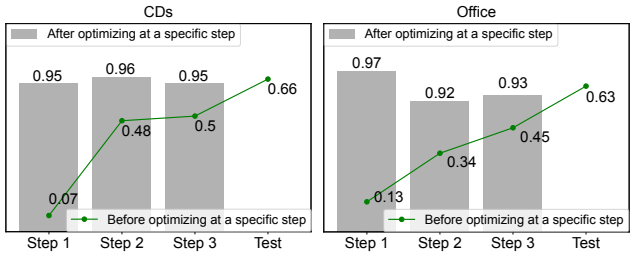

Figure 3: Performance comparison w.r.t. the progress of optimization. The Y-axis denotes the proportion of agents making accurate choices. The X-axis denotes the step of optimization. "Test" indicates the results in the test dataset.

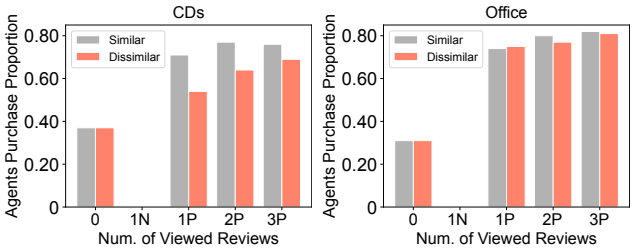

Figure 4: Proportion of agents buying items after viewing reviews. "Similar" means that the reviews are written by users with similar preferences to test users. "1N" indicates one negative review.

3.3.3 Effectiveness of Collaborative Reflection. To explore the efficacy of collaborative reflection in agent optimization, we evaluate the change in the alignment between agents and real users as optimization progresses. In this experiment, we extract the last three interactions from user behavior sequences to optimize agents, which we refer to as three optimization steps. At each step of optimization, we test whether these agents can select the positive item from two candidates before and after optimization. As illustrated in Figure 3, continuous optimization enables user agents to align their preferences with real users, leading to an increasing number making correct choices on initial attempts (*i.e.,* before optimizing at this step). Moreover, around 95% of user agents can make correct choices after collaborative reflection, highlighting its effectiveness.

3.4 Simulations on Other Types of Interactions

3.4.1 User-user Interaction Simulation. Real-world users often seek advice from others when interacting with unfamiliar items. For example, they may browse reviews on shopping websites. To explore whether user agents can exhibit similar social behaviors, we simulate user-user interaction via AgentCF, where user agents read and write reviews. Specifically, given the test items that test users haven't interacted with, we first ask other user agents, who have previously interacted with these items, to write reviews. The test users are then prompted to read these reviews and make decisions. We compare the decisions of the test users before and after viewing these reviews. As illustrated in Figure 4, the simulated user agents exhibit behaviors akin to those of real users. They show a growing inclination to purchase items upon encountering positive reviews, refrain from purchasing after encountering negative reviews, and trust reviews from users with similar preferences.

3.4.2 Item-item Interaction Simulation. Recommender systems face challenges in suggesting new items. In this part, we aim to alleviate the cold-start problem by exploring autonomous interactions between new and popular item agents. Specifically, we prompt these new item agents, equipped with identity information such as titles and categories, to interact with and learn from popular item agents that have extensive interaction records. This helps warm up cold-start item agents by estimating adopter preferences and adjusting their memory. Then we prompt user agents to rank the new items among nine other randomly sampled but well-trained items, and compare the ranking results obtained using cold-start memory and adjusted memory. As illustrated in Figure 5, the interaction process enhances the propagation of collected user preferences to new items and improves ranking performance. It indicates that item-item interactions activated by AgentCF benefit recommendations.

3.4.3 Process of Preference Propagation. As previously mentioned, our approach achieves preference propagation via collaborative optimization. Here we illustrate this process in detail. Specifically, we first initialize the memory of a seed user agent with special preference descriptions that do not spontaneously emerge during normal optimization. The user and item agents are then prompted to autonomously interact and optimize their memories. We showcase the optimized memories of several agents in Figure 6. As we can see, through collaborative optimization, user and item agents mutually interact, acting as bridges that facilitate the propagation of their preferences to other agents with similar interaction behaviors.

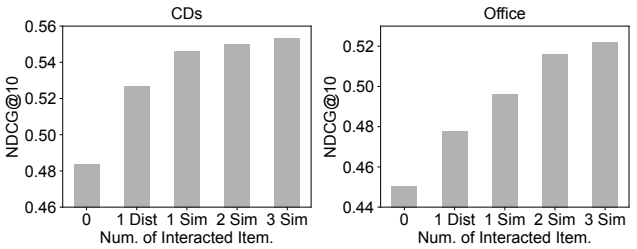

Figure 5: Performance comparisons w.r.t. cold-start item agents' interactions with different popular item agents. "Dist" means that the interacted item has distinct identity information compared to the cold-start item. "Sim" means similar.

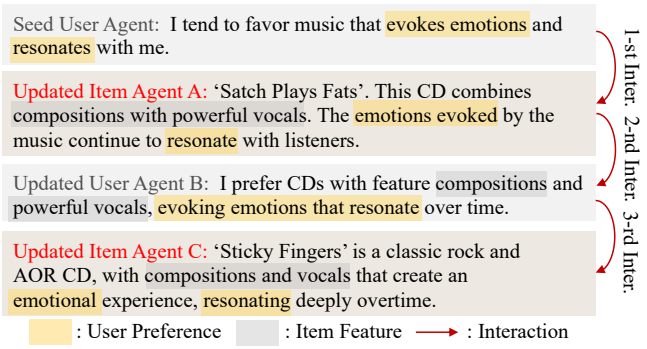

Figure 6: Case Study of preference propagation. The preferences of user and item agents are respectively propagated to other agents with similar behaviors through interactions.

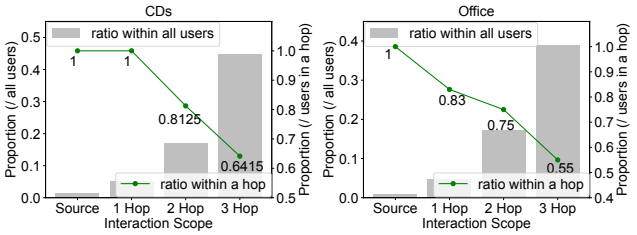

Figure 7: The proportions of user agents having seed user's preferences after continuous preference propagation.

To assess preference propagation efficiency, we ask each user agent about their possession of the seed user's preferences. The results are presented in Figure 7. We observe that through continuous interactions, an increasing number of user agents with similar behaviors to the seed user can express similar preferences. Additionally, as the interaction scope expands, the proportion of user agents expressing such preference gradually decreases. This indicates that information decay during the propagation process, in line with real-world information diffusion principles [51]. These results demonstrate the potential of our method in developing collaborative filtering systems as a cradle for potential collective intelligence.

4 RELATED WORK

LLM-powered Agents. Recently, LLM-powered agents have shown the potential to develop Artificial General Intelligence (AGI) due to their capabilities in reasoning and planning [43, 56]. Equipped with memory [24, 40] and reflection [30, 39] modules, these agents can store their past experiences and make better decisions for future behaviors. Numerous studies have been proposed to employ agents for simulating human-like interactions, including daily lives in smallville [34], debate [5, 9, 23], and even game-playing [6, 13, 41]. These studies inspire us to explore the application of LLM-powred agents in simulating user-item interactions.

Language Model for Recommendation. Numerous attempts have been made to leverage language models for solving recommendation tasks [10, 22, 25, 36, 47]. Specifically, these studies mainly prompt language models to infer user preferences based on user historical interactions [50?]. However, due to the gap between universal knowledge of LLMs and domains-specific user behavior patterns [54], these studies may fall short in providing personalized recommendations. To solve this, several studies propose to incorporate knowledge of LLMs to enhance recommendation models, rather than directly taking LLMs as recommenders [45, 48]. Some studies also propose to specialize LLMs for recommendation by fine-tuning them on recommendation data, which can be time-consuming [1, 7, 12, 26]. Recently, researchers attempt to incorporate agents into recommender systems, focusing on facilitating recommendations or user behavior simulations [19, 44, 46, 53]. However, they mainly focus on user behavior, neglecting the modeling of user-item relations, which is the core of developing recommender systems. Unlike these methods, we further propose the agent-based collaborative filtering approach, enabling user and item agents to model their two-sided relations.

5 CONCLUSION AND FUTURE WORK

In this paper, we proposed *AgentCF*, an agent-based collaborative filtering approach for simulating user-item interactions in recommender systems. We creatively consider not only users but also items as agents, allowing for the modeling of their two-sided relations through interactions of LLM-powered agents. Specifically, we develop a collaborative learning approach that optimizes both kinds of agents together, where these agents perform autonomous interactions and reflect on the disparities between their decisions and real-world interaction records. During this process, user and item agents mutually align their preferences, and propagate this information to other agents in subsequent interactions, implicitly modeling the collaborative filtering idea. The simulated agents exhibit human-like behaviors in various types of interactions, demonstrating the effectiveness of AgentCF. In the future, we will explore more types of real-world scenarios and their corresponding interactions.

ACKNOWLEDGEMENTS

This work was partially supported by National Natural Science Foundation of China under Grant No. 62222215, Beijing Natural Science Foundation under Grant No. L233008 and 4222027, Young Elite Scientists Sponsorship Program by CAST under Grant No. 2023QNRC001. Xin Zhao is the corresponding author.

REFERENCES

[1] Keqin Bao, Jizhi Zhang, Yang Zhang, Wenjie Wang, Fuli Feng, and Xiangnan He. 2023. TALLRec: An Effective and Efficient Tuning Framework to Align Large Language Model with Recommendation. *arXiv preprint arXiv:2305.00447* (2023).

[2] Léon Bottou. 2010. Large-Scale Machine Learning with Stochastic Gradient Descent. In *COMPSTAT*, Yves Lechevallier and Gilbert Saporta (Eds.). Physica-Verlag, 177–186.

[3] John S. Breese, David Heckerman, and Carl Myers Kadie. 2013. Empirical Analysis of Predictive Algorithms for Collaborative Filtering. *CoRR* abs/1301.7363 (2013).

[4] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.

[5] Chi-Min Chan, Weize Chen, Yusheng Su, Jianxuan Yu, Wei Xue, Shanghang Zhang, Jie Fu, and Zhiyuan Liu. 2023. ChatEval: Towards Better LLM-based Evaluators through Multi-Agent Debate. *CoRR* abs/2308.07201 (2023).

[6] Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chen Qian, Chi-Min Chan, Yujia Qin, Yaxi Lu, Ruobing Xie, Zhiyuan Liu, Maosong Sun, and Jie Zhou. 2023. AgentVerse: Facilitating Multi-Agent Collaboration and Exploring Emergent Behaviors in Agents. *CoRR* abs/2308.10848 (2023).

[7] Zeyu Cui, Jianxin Ma, Chang Zhou, Jingren Zhou, and Hongxia Yang. 2022. M6-Rec: Generative Pretrained Language Models are Open-Ended Recommender Systems. *arXiv preprint arXiv:2205.08084* (2022).

[8] Sunhao Dai, Ninglu Shao, Haiyuan Zhao, Weijie Yu, Zihua Si, Chen Xu, Zhongxiang Sun, Xiao Zhang, and Jun Xu. 2023. Uncovering ChatGPT's Capabilities in Recommender Systems. *arXiv preprint arXiv:2305.02182* (2023).

[9] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. 2023. Improving Factuality and Reasoning in Language Models through Multiagent Debate. *CoRR* abs/2305.14325 (2023).

[10] Wenqi Fan, Zihuai Zhao, Jiatong Li, Yunqing Liu, Xiaowei Mei, Yiqi Wang, Jiliang Tang, and Qing Li. 2023. Recommender Systems in the Era of Large Language Models (LLMs). *CoRR* (2023).

[11] Chen Gao, Xiaochong Lan, Zhihong Lu, Jinzhu Mao, Jinghua Piao, Huandong Wang, Depeng Jin, and Yong Li. 2023. S^3: Social-network Simulation System with Large Language Model-Empowered Agents. *CoRR* abs/2307.14984 (2023).

[12] Shijie Geng, Shuchang Liu, Zuohui Fu, Yingqiang Ge, and Yongfeng Zhang. 2022. Recommendation as language processing (rlp): A unified pretrain, personalized prompt & predict paradigm (p5). In *RecSys*.

[13] Ran Gong, Qiuyuan Huang, Xiaojian Ma, Hoi Vo, Zane Durante, Yusuke Noda, Zilong Zheng, Song-Chun Zhu, Demetri Terzopoulos, Li Fei-Fei, and Jianfeng Gao. 2023. MindAgent: Emergent Gaming Interaction. *CoRR* abs/2309.09971 (2023).

[14] Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. 2017. Neural Collaborative Filtering. In *WWW*, Rick Barrett, Rick Cummings, Eugene Agichtein, and Evgeniy Gabrilovich (Eds.). ACM, 173–182.

[15] Yupeng Hou, Zhankui He, Julian McAuley, and Wayne Xin Zhao. 2023. Learning vector-quantized item representation for transferable sequential recommenders. In *Proceedings of the ACM Web Conference 2023*. 1162–1171.

[16] Yupeng Hou, Shanlei Mu, Wayne Xin Zhao, Yaliang Li, Bolin Ding, and Ji-Rong Wen. 2022. Towards Universal Sequence Representation Learning for Recommender Systems. In *KDD*.

[17] Yupeng Hou, Junjie Zhang, Zihan Lin, Hongyu Lu, Ruobing Xie, Julian J. McAuley, and Wayne Xin Zhao. 2023. Large Language Models are Zero-Shot Rankers for Recommender Systems. *CoRR* (2023).

[18] Xu Huang, Jianxun Lian, Yuxuan Lei, Jing Yao, Defu Lian, and Xing Xie. 2023. Recommender AI Agent: Integrating Large Language Models for Interactive Recommendations. *CoRR* abs/2308.16505 (2023).

[19] Eugene Ie, Chih-Wei Hsu, Martin Mladenov, Vihan Jain, Sanmit Narvekar, Jing Wang, Rui Wu, and Craig Boutilier. 2019. RecSim: A Configurable Simulation Platform for Recommender Systems. *CoRR* abs/1909.04847 (2019).

[20] Wang-Cheng Kang and Julian McAuley. 2018. Self-Attentive Sequential Recommendation. In *ICDM*.

[21] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. 2023. CAMEL: Communicative Agents for "Mind" Exploration of Large Scale Language Model Society. *CoRR* abs/2303.17760 (2023).

[22] Lei Li, Yongfeng Zhang, Dugang Liu, and Li Chen. 2023. Large Language Models for Generative Recommendation: A Survey and Visionary Discussions. *CoRR* abs/2309.01157 (2023).

[23] Tian Liang, Zhiwei He, Wenxiang Jiao, Xing Wang, Yan Wang, Rui Wang, Yujiu Yang, Zhaopeng Tu, and Shuming Shi. 2023. Encouraging Divergent Thinking in Large Language Models through Multi-Agent Debate. *CoRR* abs/2305.19118 (2023).

[24] Xinnian Liang, Bing Wang, Hui Huang, Shuangzhi Wu, Peihao Wu, Lu Lu, Zejun Ma, and Zhoujun Li. 2023. Unleashing Infinite-Length Input Capacity for Large-scale Language Models with Self-Controlled Memory System. *CoRR* abs/2304.13343 (2023).

[25] Jianghao Lin, Xinyi Dai, Yunjia Xi, Weiwen Liu, Bo Chen, Xiangyang Li, Chenxu Zhu, Huifeng Guo, Yong Yu, Ruiming Tang, and Weinan Zhang. 2023. How Can Recommender Systems Benefit from Large Language Models: A Survey. *CoRR* (2023).

[26] Jianghao Lin, Rong Shan, Chenxu Zhu, Kounianhua Du, Bo Chen, Shigang Quan, Ruiming Tang, Yong Yu, and Weinan Zhang. 2023. ReLLa: Retrieval-enhanced Large Language Models for Lifelong Sequential Behavior Comprehension in Recommendation. *CoRR* abs/2308.11131 (2023).

[27] Jiaju Lin, Haoran Zhao, Aochi Zhang, Yiting Wu, Huqiuyue Ping, and Qin Chen. 2023. AgentSims: An Open-Source Sandbox for Large Language Model Evaluation. *CoRR* abs/2308.04026 (2023).

[28] Junling Liu, Chao Liu, Renjie Lv, Kang Zhou, and Yan Zhang. 2023. Is ChatGPT a Good Recommender? A Preliminary Study. arXiv:2304.10149 [cs]

[29] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Sean Welleck, Bodhisattwa Prasad Majumder, Shashank Gupta, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-Refine: Iterative Refinement with Self-Feedback. *CoRR* abs/2303.17651 (2023).

[30] Ning Miao, Yee Whye Teh, and Tom Rainforth. 2023. SelfCheck: Using LLMs to Zero-Shot Check Their Own Step-by-Step Reasoning. *CoRR* abs/2308.00436 (2023). https://doi.org/10.48550/arXiv.2308.00436 arXiv:2308.00436

[31] Jianmo Ni, Jiacheng Li, and Julian J. McAuley. 2019. Justifying Recommendations using Distantly-Labeled Reviews and Fine-Grained Aspects. In *EMNLP-IJCNLP*. 188–197.

[32] Balaji Padmanabhan and Alexander Tuzhilin. 1999. Unexpectedness as a measure of interestingness in knowledge discovery. *Decis. Support Syst.* 27, 3 (1999), 303–318.

[33] Liangming Pan, Michael Saxon, Wenda Xu, Deepak Nathani, Xinyi Wang, and William Yang Wang. 2023. Automatically Correcting Large Language Models: Surveying the landscape of diverse self-correction strategies. *CoRR* abs/2308.03188 (2023).

[34] Joon Sung Park, Joseph C. O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative Agents: Interactive Simulacra of Human Behavior. In *UIST*. ACM, 2:1–2:22.

[35] Chen Qian, Xin Cong, Cheng Yang, Weize Chen, Yusheng Su, Juyuan Xu, Zhiyuan Liu, and Maosong Sun. 2023. Communicative Agents for Software Development. *CoRR* abs/2307.07924 (2023).

[36] Zekai Qu, Ruobing Xie, Chaojun Xiao, Yuan Yao, Zhiyuan Liu, Fengzong Lian, Zhanhui Kang, and Jie Zhou. 2023. Thoroughly Modeling Multi-domain Pre-trained Recommendation as Language. *CoRR* abs/2310.13540 (2023).

[37] Steffen Rendle, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. 2009. BPR: Bayesian Personalized Ranking from Implicit Feedback. In *UAI*, Jeff A. Bilmes and Andrew Y. Ng (Eds.). AUAI Press, 452–461.

[38] Stephen E. Robertson and Hugo Zaragoza. 2009. The Probabilistic Relevance Framework: BM25 and Beyond. *Found. Trends Inf. Retr.* 3, 4 (2009), 333–389.

[39] Noah Shinn, Beck Labash, and Ashwin Gopinath. 2023. Reflexion: an autonomous agent with dynamic memory and self-reflection. *CoRR* abs/2303.11366 (2023).

[40] Shangqing Tu, Chunyang Li, Jifan Yu, Xiaozhi Wang, Lei Hou, and Juanzi Li. 2023. ChatLog: Recording and Analyzing ChatGPT Across Time. *CoRR* abs/2304.14106 (2023).

[41] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An Open-Ended Embodied Agent with Large Language Models. *CoRR* abs/2305.16291 (2023).

[42] Lei Wang and Ee-Peng Lim. 2023. Zero-Shot Next-Item Recommendation Using Large Pretrained Language Models. arXiv:2304.03153 [cs]

[43] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. 2023. A Survey on Large Language Model based Autonomous Agents. *CoRR* abs/2308.11432 (2023). https://doi.org/10.48550/arXiv.2308.11432 arXiv:2308.11432

[44] Lei Wang, Jingsen Zhang, Xu Chen, Yankai Lin, Ruihua Song, Wayne Xin Zhao, and Ji-Rong Wen. 2023. RecAgent: A Novel Simulation Paradigm for Recommender Systems. *CoRR* abs/2306.02552 (2023).

[45] Yan Wang, Zhixuan Chu, Xin Ouyang, Simeng Wang, Hongyan Hao, Yue Shen, Jinjie Gu, Siqiao Xue, James Y. Zhang, Qing Cui, Longfei Li, Jun Zhou, and Sheng Li. 2023. Enhancing Recommender Systems with Large Language Model Reasoning Graphs. *CoRR* abs/2308.10835 (2023).

[46] Yancheng Wang, Ziyan Jiang, Zheng Chen, Fan Yang, Yingxue Zhou, Eunah Cho, Xing Fan, Xiaojiang Huang, Yanbin Lu, and Yingzhen Yang. 2023. RecMind: Large Language Model Powered Agent For Recommendation. *CoRR* abs/2308.14296 (2023).

[47] Likang Wu, Zhi Zheng, Zhaopeng Qiu, Hao Wang, Hongchao Gu, Tingjia Shen, Chuan Qin, Chen Zhu, Hengshu Zhu, Qi Liu, Hui Xiong, and Enhong Chen. 2023. A Survey on Large Language Models for Recommendation. *CoRR* (2023).

[48] Yunjia Xi, Weiwen Liu, Jianghao Lin, Jieming Zhu, Bo Chen, Ruiming Tang, Weinan Zhang, Rui Zhang, and Yong Yu. 2023. Towards Open-World Recommendation with Knowledge Augmentation from Large Language Models. *CoRR* abs/2306.10933 (2023).

[49] Lanling Xu, Zhen Tian, Gaowei Zhang, Lei Wang, Junjie Zhang, Bowen Zheng, Yifan Li, Yupeng Hou, Xingyu Pan, Yushuo Chen, et al. 2022. Recent Advances in RecBole: Extensions with more Practical Considerations. *arXiv preprint arXiv:2211.15148* (2022).

[50] Lanling Xu, Junjie Zhang, Bingqian Li, Jinpeng Wang, Mingchen Cai, Wayne Xin Zhao, and Ji-Rong Wen. 2024. Prompting Large Language Models for Recommender Systems: A Comprehensive Framework and Empirical Analysis. *CoRR* abs/2401.04997 (2024).

[51] Jaewon Yang and Jure Leskovec. 2010. Modeling Information Diffusion in Implicit Networks. In *ICDM*, Geoffrey I. Webb, Bing Liu, Chengqi Zhang, Dimitrios Gunopulos, and Xindong Wu (Eds.). IEEE Computer Society, 599–608.

[52] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *ICLR*. OpenReview.net.

[53] An Zhang, Leheng Sheng, Yuxin Chen, Hao Li, Yang Deng, Xiang Wang, and Tat-Seng Chua. 2023. On Generative Agents in Recommendation. *CoRR* abs/2310.10108 (2023).

[54] Junjie Zhang, Ruobing Xie, Yupeng Hou, Wayne Xin Zhao, Leyu Lin, and Ji-Rong Wen. 2023. Recommendation as Instruction Following: A Large Language Model Empowered Recommendation Approach. *CoRR* abs/2305.07001 (2023).

[55] Wayne Xin Zhao, Shanlei Mu, Yupeng Hou, Zihan Lin, Yushuo Chen, Xingyu Pan, Kaiyuan Li, Yujie Lu, Hui Wang, Changxin Tian, et al. 2021. Recbole: Towards a unified, comprehensive and efficient framework for recommendation algorithms. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*. 4653–4664.

[56] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. 2023. A survey of large language models. *arXiv preprint arXiv:2303.18223* (2023).

[57] Kun Zhou, Hui Wang, Wayne Xin Zhao, Yutao Zhu, Sirui Wang, Fuzheng Zhang, Zhongyuan Wang, and Ji-Rong Wen. 2020. S3-Rec: Self-Supervised Learning for Sequential Recommendation with Mutual Information Maximization. In *CIKM*. 1893–1902.

A IMPLEMENTATION DETAILS

We implement AgentCF and other baselines based on a popular open-source recommendation framework RecBole [49, 55]. We implement LLMRank using gpt-3.5-turbo-16k-0613. The autonomous item selection component of AgentCF is implemented using text-davinci-003, while the collaborative reflection and inference components of AgentCF are implemented using gpt-3.5-turbo-16k-0613. The hyperparameter temperature, used when calling APIs of LLMs, is set to 0.

B PROMPT DISIGN

The prompt of the collaborative reflection process consists of three main components: (1) Reflecting on the reasons behind inconsistent decisions compared to real users. (2) Exploring new preferences and dislikes based on the features of positive and negative items. (3) Removing outdated or inconsistent content from their previous short-term memory. By integrating these components, the user agents can be prompted to correct the misleading conceptions in their memories and enhance their alignment with real users. The user and item agents can also better understand their two-sided relations by discriminating between positive instances and negative instances. At each step of interaction, the autonomous item selection process and the collaborative reflection are iterated until user agents make consistent choices with real-world interaction records or reach the maximum round of iterations. The maximum rounds of optimization iterations per step is set to 2.

If a user agent makes the right choice that aligns with the real behavior, it indicates that the simulated user agent and item agents have captured their two-sided relations and exhibit a strong alignment with real-world individuals (at least in this optimization step). In such cases, we inform them about the correctness and store the related interaction information in their memory. The prompt in these instances consists of two main components: (1) Exploring new preferences and dislikes based on the features of positive and negative items. (2) Removing outdated or inconsistent content from their previous short-term memory. In other words, we no longer prompt agents to conduct reflections.

B.1 Agent-based Collaborative Filtering

Initialization

• User Agent

I enjoy listening to CDs very much.

• Positive Item Agent

The CD is called "Brainwashed". The category of this CD is: "Classic Rock; Album-Oriented Rock (AOR)".

• Negative Item Agent

The CD is called "O, Yeah! Ultimate Aerosmith Hits". The category of this CD is: "Classic Rock; Album-Oriented Rock (AOR)".

"Forward" Autonomous Interaction • User Agent$_{\text{system prompt}}$

You are a CD enthusiast. Here is your self-introduction, expressing your preferences and dislikes: "$user agent memory". Now, you are considering to select a CD from two candidate CDs. The features of these two candidate CDs are listed as follows: "1. $neg item agent memory \n 2. $pos item memory". \n \n Please select the CD that aligns best with your preferences. Furthermore, you must articulate why you've chosen that particular CD while rejecting the other. \n To do this, please follow these steps:\n 1. Extract your preferences and dislikes from your self-introduction.\n 2. Evaluate the two candidate CDs in light of your preferences and dislikes. Make your choice by considering the correlation between your preferences/dislikes and the features of the CDs.\n 3. Explain why you make such choices, from the perspective of the relationship between your preferences/dislikes and the features of these candidate CDs. \n \n Important note:\n 1. Your output should in the format: Chosen CD: [Title of the selected CD] \n Explanation: [Detailed rationale behind your choice and reasons for rejecting the other CD].\n 2. When identifying your likes and dislikes, don't fabricate them! If your self-introduction doesn't specify any relevant preferences or dislikes, use common knowledge to inform your decision, such as the popularity of the CDs. \n 3. you must choose one of these two candidates, instead of choosing both of them. \n 4. Your explanation needs to be comprehensive and specific. A general preference, such as a certain genre of CD, is insufficient. Your reasoning should delve into the finer attributes of the CD.\n 5. Base your explanation on facts. For instance, if your self-introduction doesn't reveal any specific preferences, you cannot assert that your decision was influenced by such preferences."

• User Agent$_{\text{response}}$

Chosen CD: O, Yeah! Ultimate Aerosmith Hits \n Explanation: I chose O, Yeah! Ultimate Aerosmith Hits because it is a classic rock album from the Album-Oriented Rock (AOR) genre, which is a genre I enjoy listening to. The album contains some of Aerosmith's greatest hits, which I believe will provide me with a great listening experience. In comparison, Brainwashed is also a classic rock album from the AOR genre, but it is not as popular as O, Yeah! Ultimate Aerosmith Hits, and therefore I believe it will not provide me with the same level of listening experience.

"Backward" Collaborative Memory-based Optimization

• User Agent$_{\text{system prompt}}$

You are a CD enthusiast. \n Here is your previous self-introduction, exhibiting your past preferences and dislikes:"$user agent memory". Recently, you considered to choose one CD from two candidates. The features of these two candidate CDs are listed as follows:"1. $neg item agent memory \n 2. $pos item memory" . \n \n After comparing these two candidates based on your preferences and dislikes, you selected "$neg item title" to listen to while rejected the other one. You provided the following explanations for your choice, revealing your previous judgment about your preferences and dislikes for these two CDs: "$user explanation". However, upon actually listening to these two CDs, you discovered that you don't like the CD that you initially chose (Here And There Remastered). Instead, you prefer the CD that you did not choose before (Thriller).\n This indicates that you made an incorrect choice, and your judgment about your preferences and dislikes, as recorded in your explanation, was mistaken. It's possible that your preferences and dislikes, described in your previous self-introduction, have either changed or were uninformative. Therefore, your task now is to update your self-introduction, by incorporating your new preferences and dislikes revealed in this interaction. To do this, please follow these steps: 1. Analyze the misconceptions in your previous judgment about your preferences and dislikes, as recorded in your explanation, and correct these mistakes \n 2. Explore your new preferences based on the features of CDs you really enjoy (Thriller), and determine your dislikes based on the features of CDs you truly don't enjoy (Here And There Remastered). \n ... 3. Summarize your past preferences and dislikes from your previous self-introduction. Combine your newfound preferences and dislikes with your past preferences and dislikes. Filter and Remove any conflicting or repetitive parts in your past descriptions that contradict your current preferences and dislikes. \n 4. Update your self-introduction. Please start by describing your newfound preferences reflected in this interaction. Then describe

your past preferences summarized and filtered from your previous self-introduction. Afterward, please describe your dislikes. \n \n Important note:\n 1. Your output should use the following format: "My updated self-introduction: [Please write your updated self-introduction here]." \n 2. Keep your updated self-introduction under 180 words. \n 3. Any overall assessments or summarization in your self-introduction are forbidden.\n 4. You updated self-introduction only need to describe the features of CDs you prefer or dislike, without mentioning your wrong choice or your thinking process in updating your self-introduction. \n 5. You self-introduction should be specific and personalized. Any preferences and dislikes that cannot distinguish you from others are not worth recording.

• **User Agent$_{\text{response}}$**

My updated self-introduction: I enjoy listening to CDs that fall under the classic rock and album-oriented rock (AOR) genres. I particularly appreciate CDs that offer a unique and distinct sound within these genres. I find myself drawn to CDs that showcase a variety of hits and popular songs, as they provide an enjoyable listening experience. Additionally, I have discovered a newfound preference for CDs that have a more experimental and innovative approach to classic rock. These CDs offer a refreshing twist on the genre and keep me engaged throughout. On the other hand, I tend to dislike CDs that lack originality and fail to bring something new to the table. CDs that rely heavily on generic rock sounds and do not offer any standout tracks or moments tend to leave me uninterested. Overall, I seek CDs that captivate me with their creativity and deliver a memorable listening experience.

• **Pos Item Agent$_{\text{system prompt}}$**

Here is the self-introduction of a user, exhibiting his preferences and dislikes: "$user agent's memory".\n Recently, the user browsed a shopping website and considered to choose one CD from two candidate CDs.\nThe descriptions of the two candidate CDs are as follows:\n "1.$pos item agent's memory \n 2. $neg item agent's memory " \n\n By considering his preferences and dislikes from his self-introduction, he chose "$neg item title" to listen to while rejected the other one. He provided the following explanations for his choices: "$user explanation". \n \nHowever, the user found that he actually doesn't enjoy the CD that he previously chosen. Conversely, he finds the CD that he didn't choose before, titled "$pos item title", more appealing. This indicates that he made an unsuitable choice. This may be because the user was misled by the descriptions of these two candidate CDs. Actually, the user prefers a CD ("$pos item title") because its features aligns with his preferences and go against his dislikes, while he dislikes the other CD ("$neg item title") because it possesses features that he disliked. Therefore, your task is to update the descriptions of these two candidate CDs. \n To do this, follow these steps:\n 1. Analyze which features recorded in the previous CD description led to the unsuitable choice based on the user's explanation, and modify them accordingly. \n 2. Analyze the user's preferences and dislikes from his self-introduction. Explore the user preferred CD's new features that aligns with the user's preferences and goes against with his dislikes. Explore the user disliked CD's new features that aligns with the user's dislikes and goes against with his preferences. In this case, the differences between the descriptions of these two items should be amplified. Your exploration should be thorough and detailed, rather than just superficial. For example, you can't simply say a CD's genre aligns with or goes against the user's preferences, as that is too broad.\n 3. Incorporate the item's newfound features with the into its previous description. Since the previous descriptions of the candidate CDs are based on evaluations from other users, it is also worth preserving valuable content. Please try to maintain conciseness while preserving the information as much as possible.\n\n Important notes: \n 1. Your output should in the following format: "The updated description of the first CD is: [updated description]. \n The updated description of the second CD is: [updated description].". \n 2. Each updated CD description can not exceed 50 words!!! \n 3. In your updated CD description, considering that it will be visible to other users, it is advisable to utilize user preferences or dislikes when referring to a group of users collectively, rather than using "the user" to specifically denote an individual. For instance, you can mention "the user with ... preferences/dislikes".\n 4. The updated CD description should not contradict the inherent characteristics of this CD. For example, you can not say that a hip-hop CD has a very soothing rhythm. \n 5. The updated CD description should be specific, containing only the distinguishing features that differentiate this CD from others.

• **Pos Item Agent$_{\text{response}}$**

"Brainwashed" is a classic rock album from the Album-Oriented Rock (AOR) genre that breaks new ground with its experimental and innovative approach. This CD offers a refreshing twist on classic rock, delivering a captivating listening experience that keeps you engaged from start to finish. "Brainwashed" stands out from generic rock sounds, providing standout tracks and moments that showcase its originality and creativity. For the user who seeks CDs that push boundaries and bring something new to the table, "Brainwashed" is a must-listen.